Sorting

arrangement of data in particular order on the basis of some parameter

2	3	9	12	17	19

sorted in asc order (magnitude)

19	6	5	2	-1	-19

sorted in desc order (magnitude)

1	13	9	6	12
1	2	3	4	6

count of factors

sorted in asc order of count of factors

why sorting? searching becomes easier

How ~~to~~ sort? — advanced

inbuilt sort function

sort( ); → T.C: $O(n \log_2 n)$ ~~why?~~

n^2

↓

$n \log n$ — Always try to think about sorting

Q array of N integers: You have to delete all the elements from the array!

you have to pay some cost to delete an element!

cost = $\boxed{\text{sum of all elements left in array}}$

Find the min cost!

A: [~~2~~ ~~1~~ 4]

delete 1	2+1+4	= 7
delete 2	2+4	= 6
delete 4	4	= 4
		17

delete 4	4+2+1	= 7
delete 2	2+1	= 3
delete 1	1	= 1
		11

A: [4 ~~6~~ 1]

delete 6	4+6+1	= 11
delete 4	4+1	= 5
delete 1	1	= 1
		17

[3 5 1 -3]

delete 5	3+5+1+-3	= 6
delete 3	3+1-3	= 1
delete 1	1+(-3)	= -2
delete -3	-3	= -3
		2

[a b c d]

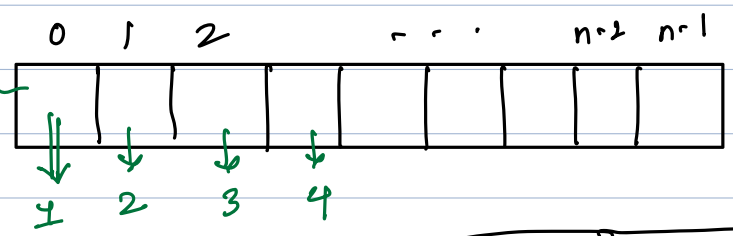
Remove a		a + b + c + d
Remove b	+	b + c + d
Remove c	+	c + d
Remove d	+	d
		a + 2b + 3c + 4d

$a > b > c > d$

start removing from largest end!

sort(arr); $n \log_2 n$

largest



desc order

$i \rightarrow (i+1)$ times

1) sort the data in desc order → S.C: O(N)

'sort'

{depend on 'sort' algo}

```
ans = 0
for ( i = 0; i < n; i++)
    ans += (i+1) * arr[i];
```

$\boxed{\text{T.C: } O(n \log_2 n)}$

S.C: ~~O(1)~~ O(N)

Q Find count of Noble integers!

array of size N

distinct elements

A[i] is noble if

count of elements smaller than A[i]	=	A[i]

0	1	2	3	4	5
1	-5	3	5	-10	4
↓	↓	↓	↓	↓	↓
2	1	3	5	0	4

ans = 3

0	1	2	3
-3	0	2	5
↓	↓	↓	↓
0	1	2	3

ans = 1

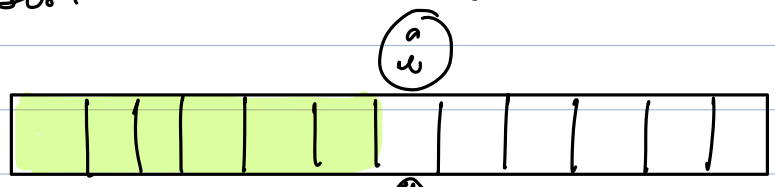
Basic way!

For every element count smaller elements

T.C: $O(N^2)$

```
ans=0;
for(i=0 ⟶ n)
{
    cnt=0;
    for(j=0; j<n; j++)
        if(arr[j] < arr[i])
            cnt++;
    if(cnt == arr[i]) ans++;
}
```

sort the data (asc)



$0 \rightarrow (i-1)$

$(i-1) - 0 + 1$

smaller elements ? $\rightarrow$ i

0	1	2	3
-3	0	2	5
↓	↓	↓	↓
0	1	2	3

T.C: $(n \log_2 n)$

```
// sort the data
// for(i=0; i<n; i++)
    if (arr[i] == i)
        ans++;
```

count of smaller elements =

what if there are duplicates ?

0	1	2	3	4
-10	1	1	3	100
↓	↓	↓	↓	↓
0	1	1	3	4

ans = 3

0	1	2	3	4	5	6	7	8
-10	1	1	2	4	4	4	8	10
↓	↓	↓	↓	↓	↓	↓	↓	↓
0	1	1	3	4	4	4	7	8

ans = 5

ans = 7

0	1	2	3	4	5	6	7	8	9	10	11	12	13
-3	0	2	2	5	5	5	5	8	8	10	10	10	14
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
0	1	2	2	4	4	4	4	8	8	10	10	10	13

If current element != prev element
count = index;

// sort the data → cnt = 0; → count of small elements

if (arr[0] == cnt) ans++;

T.C: $O(n \log_2 n)$

```
for( int i=1; i<n; i++)
{
    if ( arr[i] != arr[i-1])
        cnt = i;
    if ( cnt == arr[i])
        ans++;
}
```

10:30

sort the data in asc order of count of factors, (asc)
if count of factors are equal then sort on the basis of magnitude. (asc)

9	3	10	6	4	⟹	3	4	9	6	10
↓	↓	↓	↓	↓		↓	↓	↓	↓	↓
3	2	4	4	3		2	3	3	4	4

sort( ) — By default asc order of magnitude

sort(_ , _ , _) → comparator fn

start end

rules of your sorting

$x < y$ asc mg

↓

x should come first

x, y

count-f → cnt of fa

count-f(x) < count-f(y)

x should come first

count-f(x) > count-f(y)

y should com first

count-f(x) == count-f(y)

x <= y

x should com

x > y

y should

sort a vec

if first argument should coms first in sorted data

↓

return true

else

return false

```
bool comp( int x , int y )
{
    int cntx = count-factors(x);
    int cnty = count-factors(y);

    if ( cntx < cnty )
        return true;
    else if ( cntx > cnty)
        return false;
    else {
        if ( x <= y )
            return true;
        else
            return false;
}
```

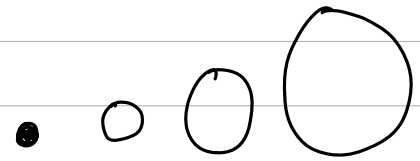
data type of elements you are sorting

C++, java

comp

asc order of size

sorting algo



sort in <u>desc order</u> of <u>magnitude</u>

```
bool comp( int x, int y)
{
        if ( x >= y)
            return true;
        else
            return false;
}
```

array of <u>strings</u> sort acc. to the length of strings in asc order!

```
bool comp( string x, string y)
{
        if( x.length < y.length)
            return true;
        else
            return false;
}
```

Doubts

N

ME

$N/2 + 1$

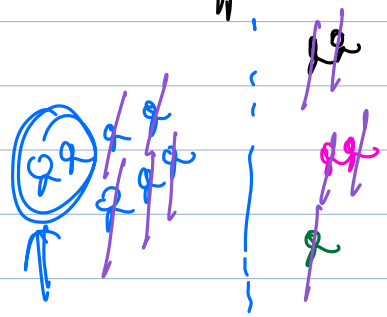
NME

$N/2 - 1 - 1$

N

NME NME

~~ME~~ ~~NME~~

$\frac{(N-2)}{2} + 1$

$= N/2 + \cancel{2}/\cancel{2} + \cancel{1}$